Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Source Code: `https://github.com/Lunarya-git/Deep-Learning-Experiments`

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224.
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

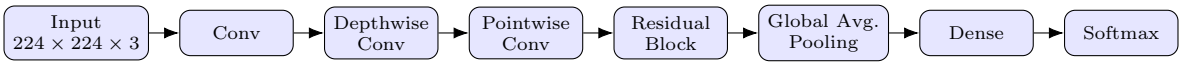
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

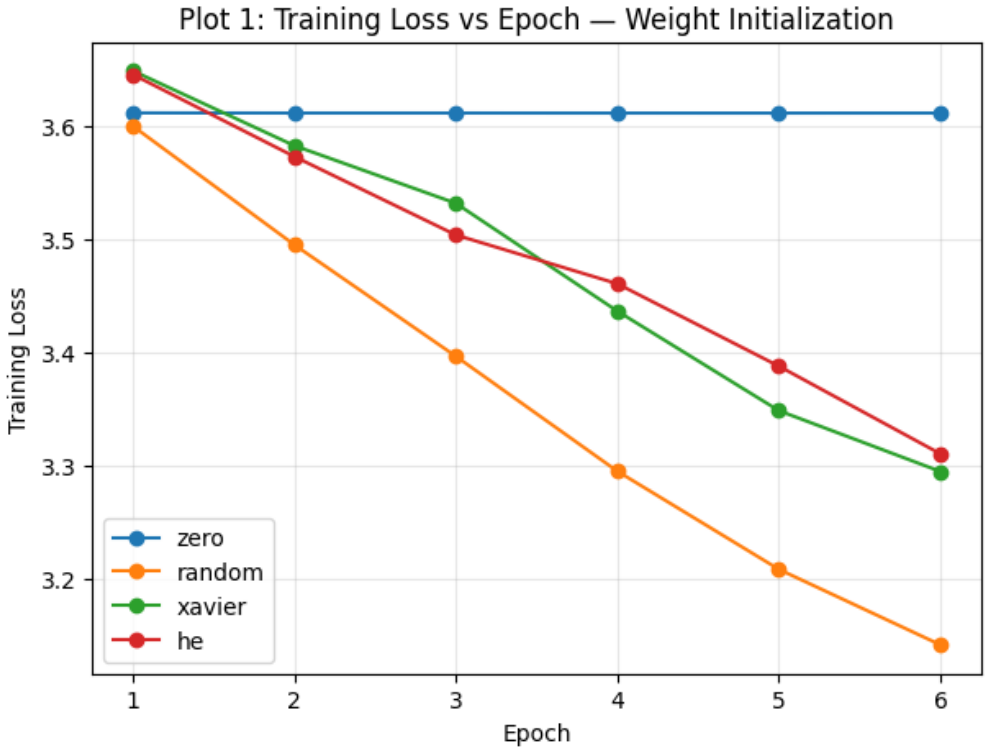
Plot 2: Validation Accuracy vs. Epoch

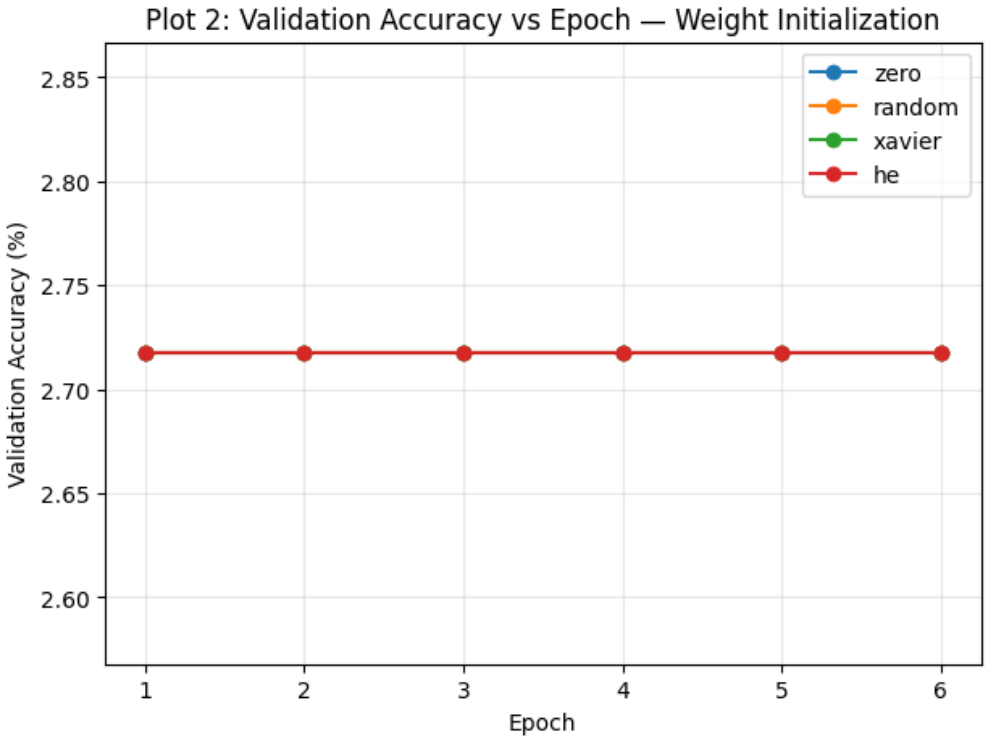
- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

Results

For this section, MobileNetV2 was trained completely from scratch (`weights=None`), so the network sees the 37-class pet classification problem for the first time, and every kernel is re-initialized with the strategy being tested. The model was trained for 6 short epochs for each of the four strategies.





Inference (Plots 1 & 2): Plot 1 shows the training loss for each initialization scheme. Zero initialization stays completely flat at loss ≈ 3.61 because every neuron in a layer receives the exact same gradient and the network can never break this symmetry, so it never really learns anything. Random, Xavier and He initialization all bring the loss down steadily, with random and He initialization dropping the fastest of the three. Plot 2, however, shows validation accuracy stuck at the same $\approx 2.7\%$ for all four methods (which is basically the accuracy of guessing one out of 37 classes at random) – 6 epochs of training a CNN from scratch is nowhere near enough for the model to generalize to unseen images, even though the training loss is falling. This is exactly the practical motivation for using transfer learning in the rest of the experiment: a pretrained network already has good features, so it does not need to be trained from a bad, symmetric starting point for a long time.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.
Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

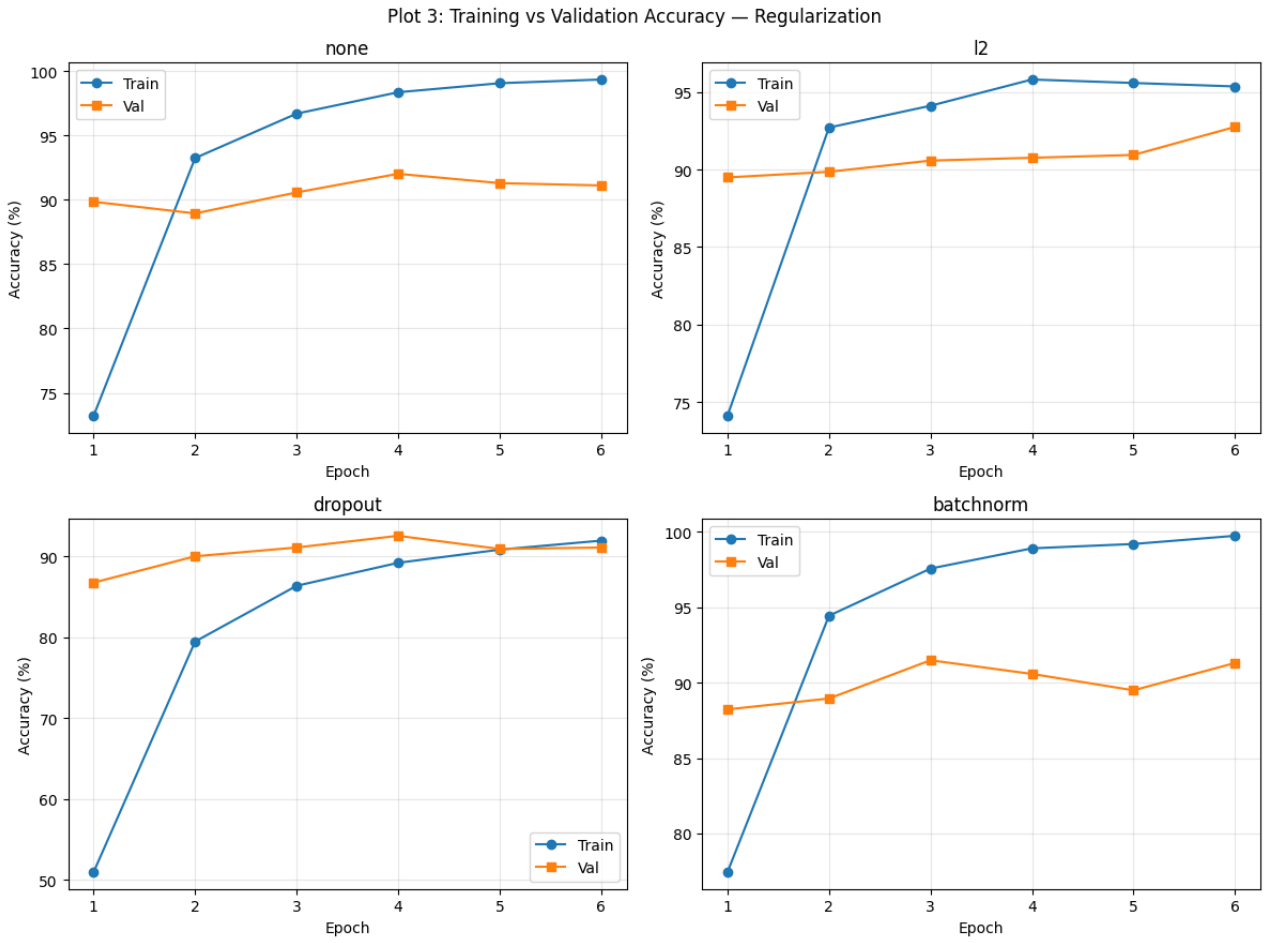
Plot 4: Training and Validation Loss vs. Epoch

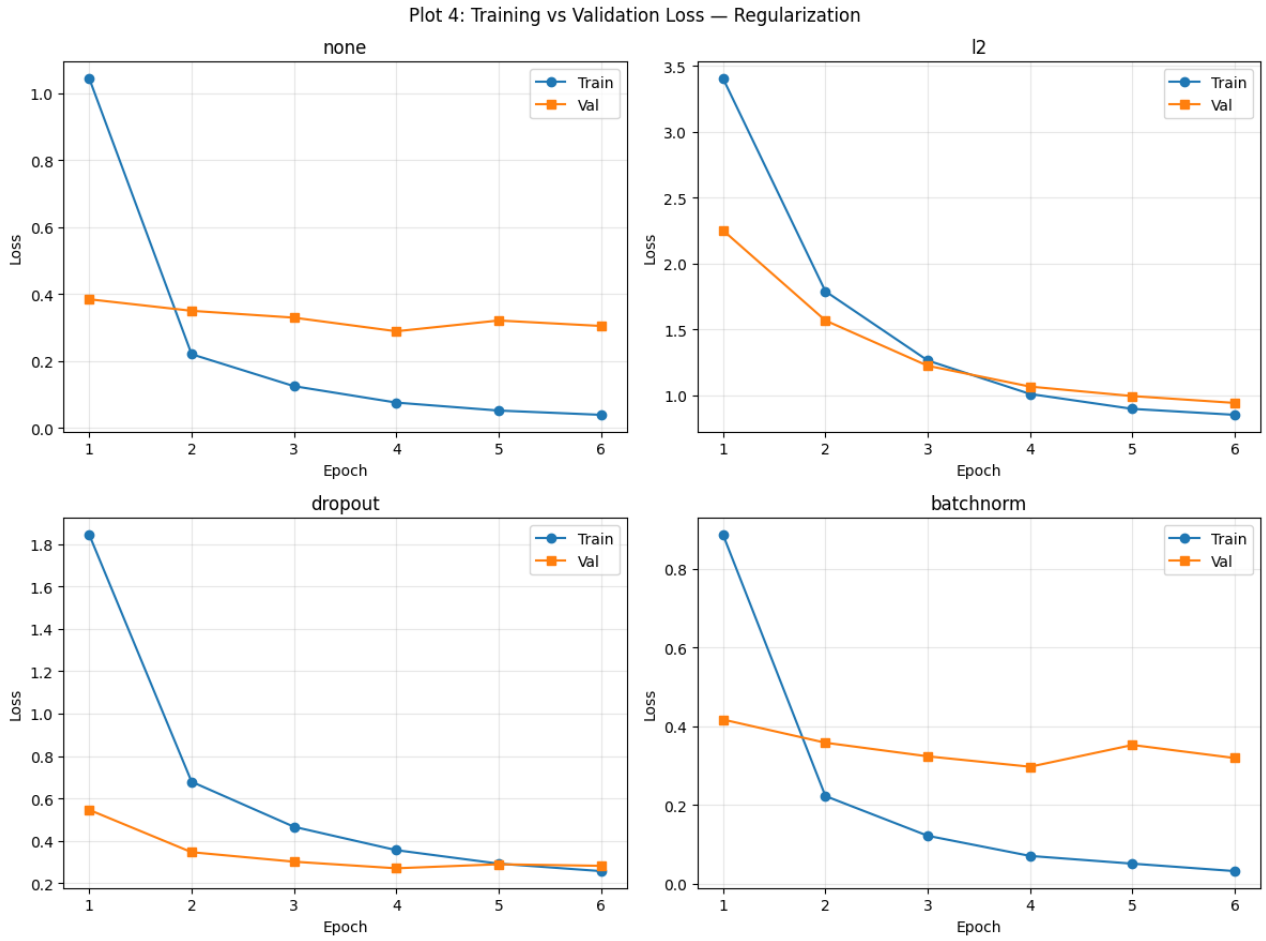
- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

Results

From this section onward, MobileNetV2 pretrained on ImageNet is used with the convolutional base **frozen**, and only the new classification head is changed and trained (no regularization, L2, Dropout, Batch Normalization).





Inference (Plots 3 & 4): In all four configurations, training accuracy keeps climbing towards 95–100% while validation accuracy flattens out around 89–92%, so there is a clear but fairly small generalization gap in every case, which makes sense because the base is frozen and only a small head is being trained. The "none" and "batchnorm" heads show the widest gap (training accuracy close to 100% versus validation around 91%), while "dropout" keeps training accuracy a little lower and closer to the validation curve, showing that Dropout is doing its job of holding back the head from memorizing the training set. In Plot 4, the "l2" configuration starts at a much higher loss because of the added L2 penalty term, but its train/validation loss curves stay closest together among all four, indicating that L2 gives the smoothest, most stable regularization of the loss surface in this run.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.
For a mini-batch

$$x_1, x_2, \ldots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

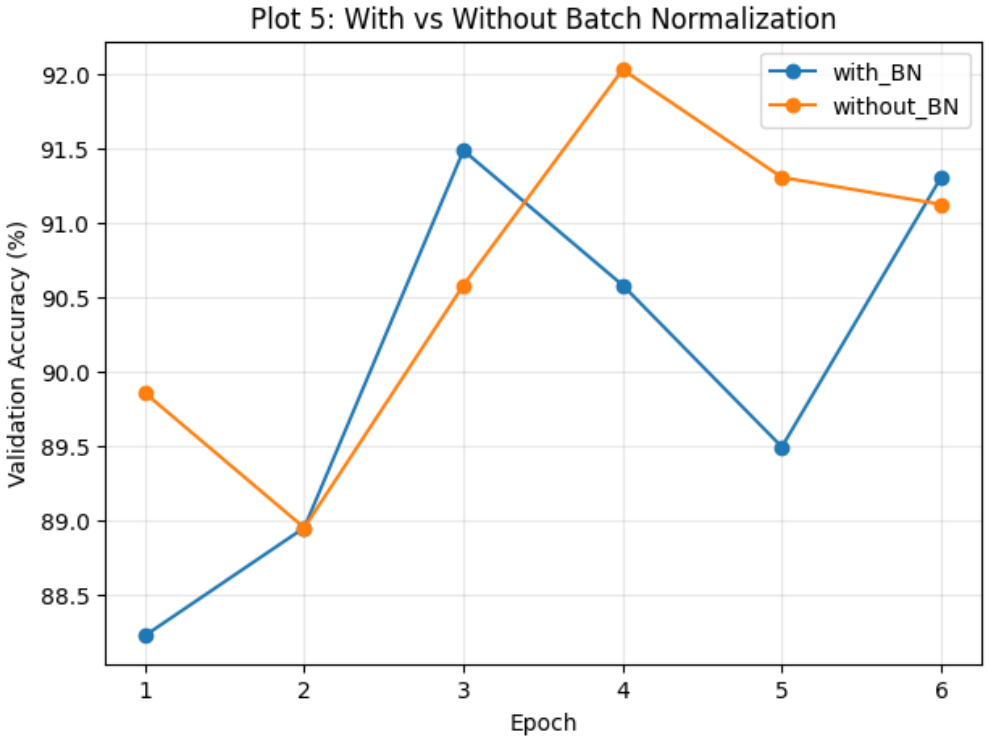
$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.



Inference (Plot 5): The "with BN" and "without BN" curves stay very close to each other throughout training, both ending up around 91–91.5% validation accuracy, with the "without BN" curve actually reaching a slightly higher peak at epoch 4 ($\approx 92\%$). This is expected here because the convolutional base (which contains most of the batch-norm layers of MobileNetV2) is frozen – only the small classifier head changes between the two runs, so adding one more batch-norm layer in the head has only a marginal effect. Batch Normalization is expected to matter far more when training deeper layers from scratch or during fine-tuning, where it stabilizes the activation statistics as the convolutional weights themselves are being updated.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

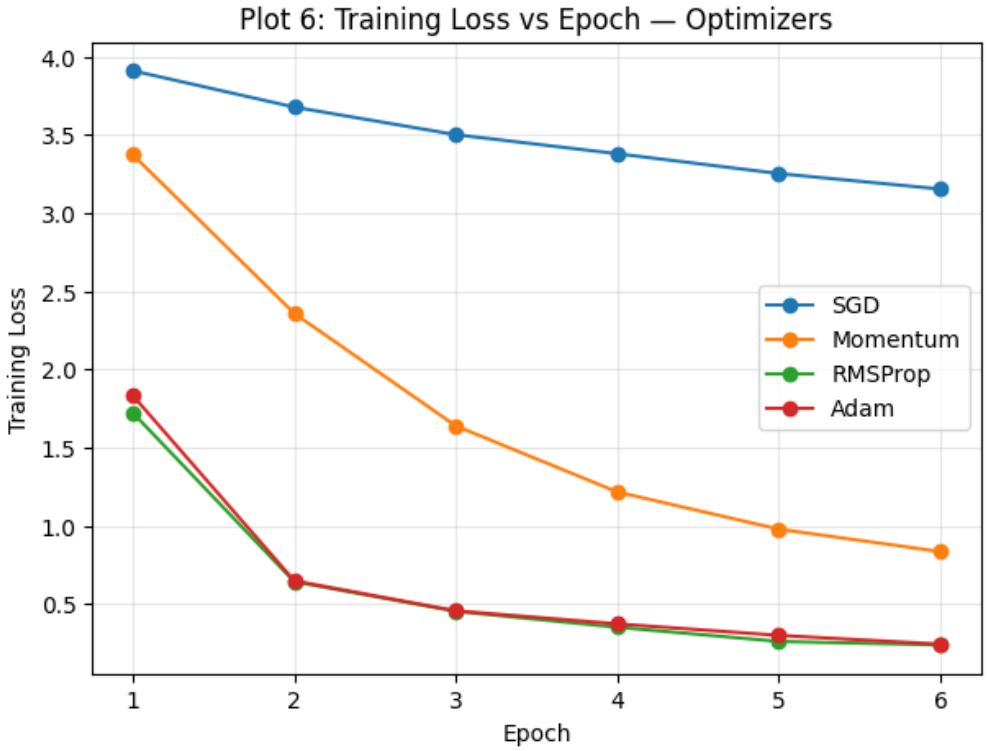
The objective is to study convergence speed, stability and final validation performance.

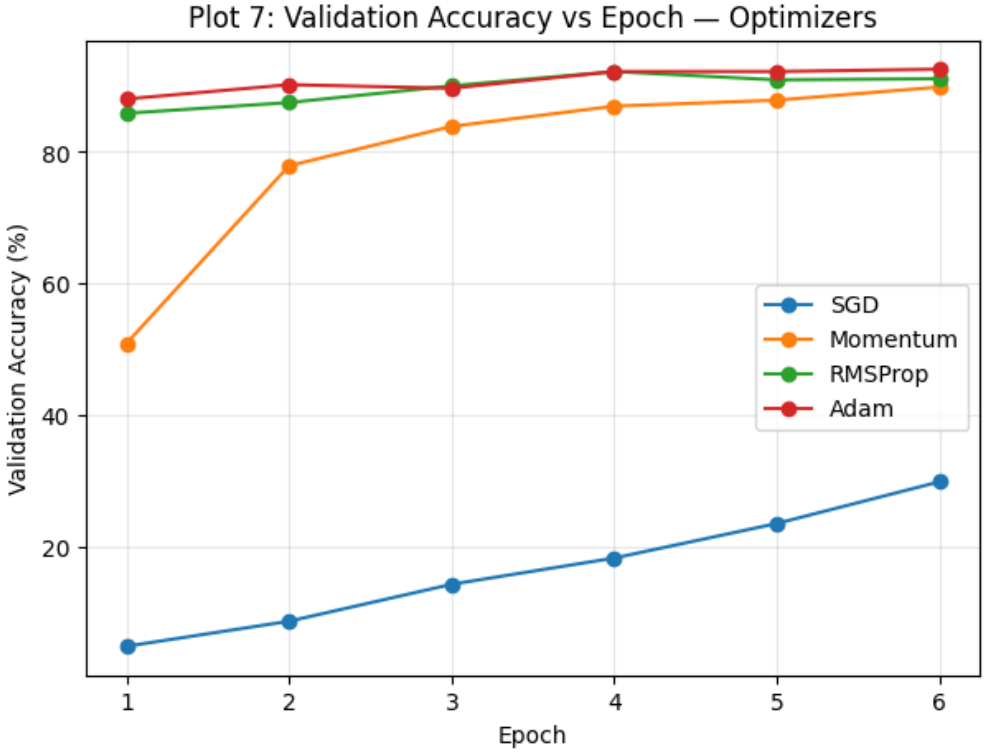
Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.





Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time (s)
SGD	3.1526	29.89%	6	38.1
Momentum	0.8350	89.86%	6	38.2
RMSProp	0.2386	92.21%	4	38.5
Adam	0.2433	92.57%	6	38.9

Inference (Plots 6 & 7, optimizer table): Plain SGD is by far the slowest – its training loss barely moves and validation accuracy crawls up to only $\approx 30\%$ in 6 epochs, because with no momentum term it takes tiny, noisy steps at this learning rate. Adding momentum helps a lot (val. accuracy jumps to $\approx 90\%$), but it is still clearly behind RMSProp and Adam in the early epochs. RMSProp and Adam both use per-parameter adaptive learning rates, so they converge the fastest and reach the highest validation accuracy (92.2% and 92.6% respectively) within just 4–6

epochs. Overall, Adam gives the best accuracy in this run while RMSProp converges a step earlier, which matches the general expectation that adaptive optimizers train noticeably faster than vanilla or momentum-based SGD on this kind of transfer-learning task.

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	$10^{-4}, 10^{-5}$
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

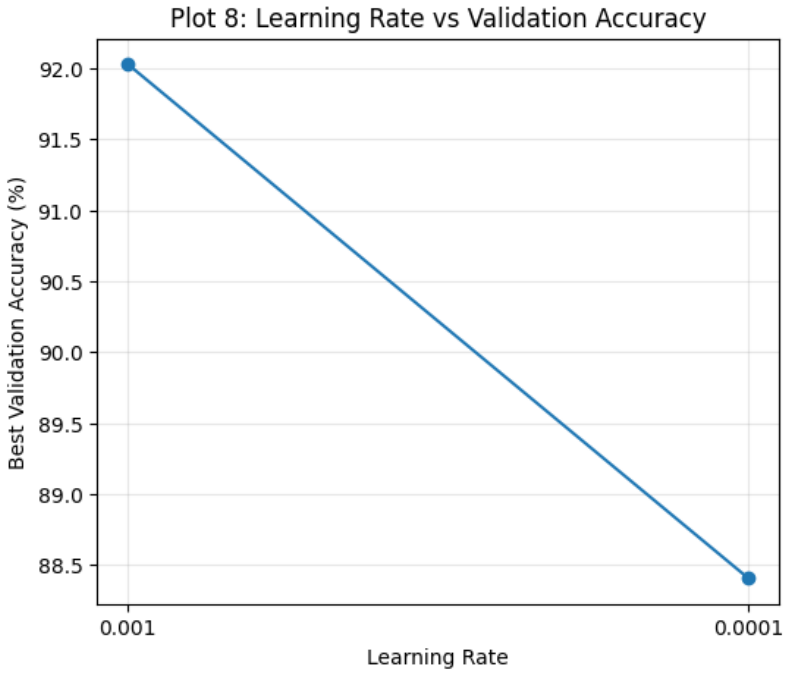
- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Plot 9: Batch Size vs. Validation Accuracy

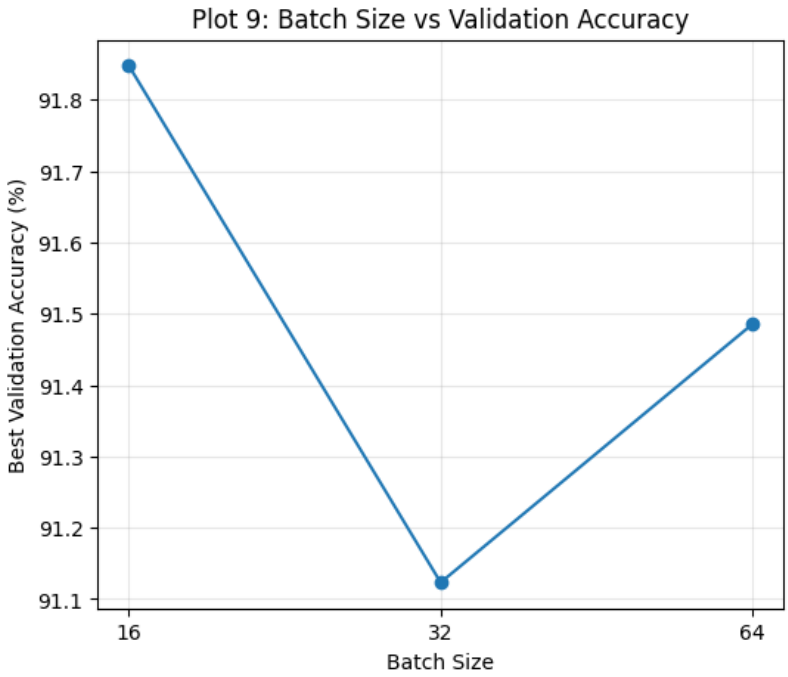
- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

Plot 10: Dropout Rate vs. Validation Accuracy

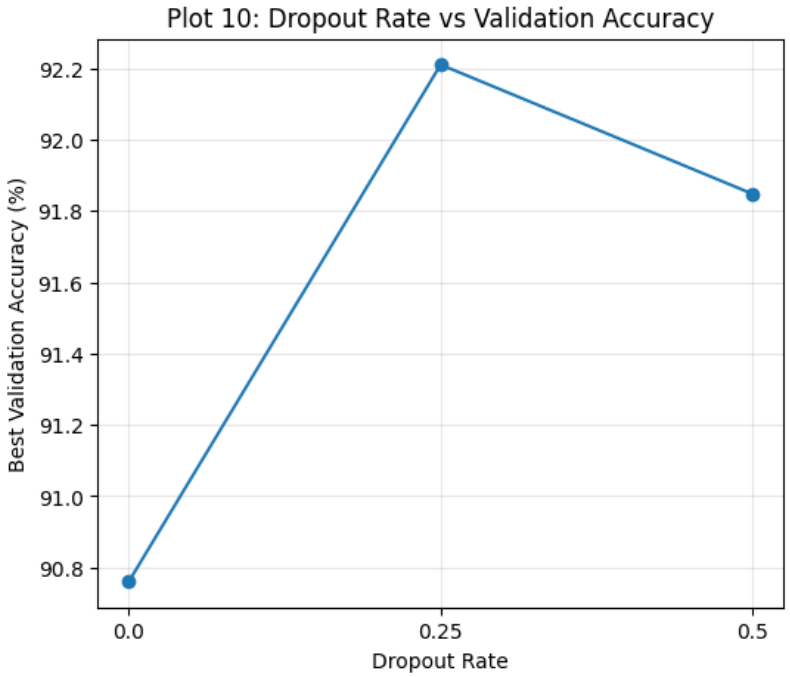
- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)



Inference (Plot 8): Best validation accuracy drops from 92.03% at learning rate 0.001 to 88.41% at learning rate 0.0001. A learning rate that is ten times smaller makes the optimizer take much smaller steps, so within the same small number of epochs the head simply does not converge as far – it would probably catch up if given more epochs, but under a fixed, short training budget the larger of the two rates is clearly better here.



Inference (Plot 9): Batch size 16 gives the best validation accuracy (91.85%), batch size 32 dips slightly to 91.12%, and batch size 64 recovers a little to 91.49%. The differences are small (well under 1%), so batch size is not a very sensitive hyperparameter for this particular head/dataset combination, though the smallest batch size does provide slightly noisier, more frequent gradient updates which seem to help a bit in this short training window.



Inference (Plot 10): Validation accuracy is lowest with no dropout at all (90.76%), peaks at a moderate dropout of 0.25 (92.21%), and drops slightly at a heavier dropout of 0.5 (91.85%). This is the classic dropout trade-off: a little dropout regularizes the head and improves generalization, but too much dropout throws away so many activations during training that the network under-fits slightly instead.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

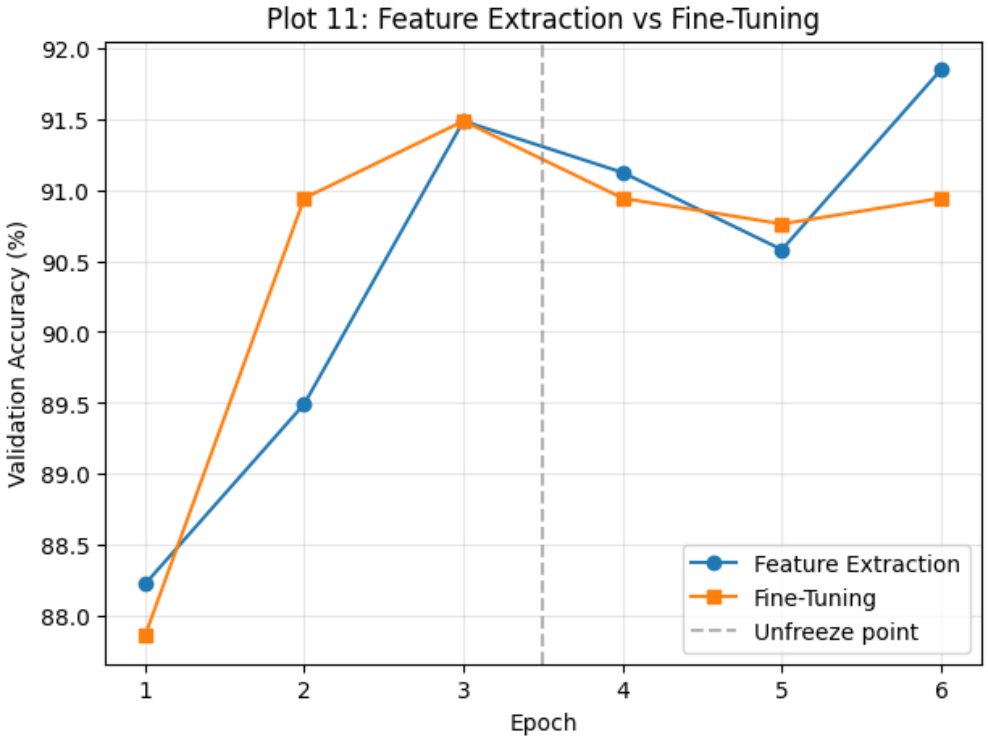
The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

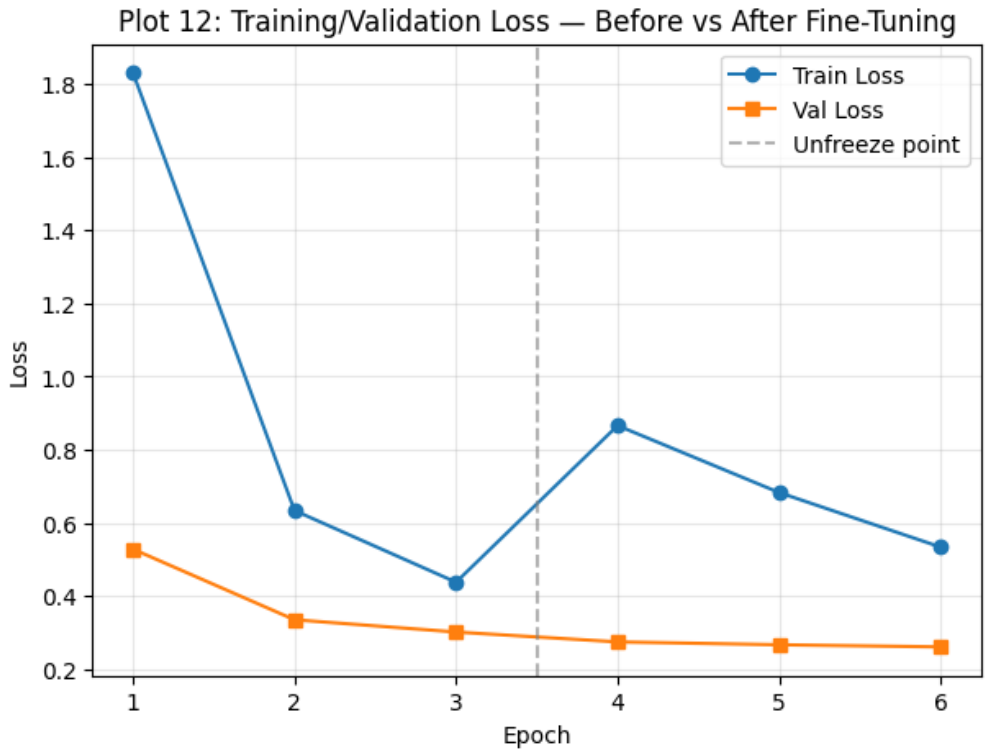
Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.





Discussion: Fine-tuning normally uses a much smaller learning rate because the convolutional base is no longer a randomly initialized set of weights – it is pretrained on ImageNet and already holds useful, general-purpose features (edges, textures, shapes). If a large learning rate is used once these layers are unfrozen, the big gradient updates can quickly wreck those useful pretrained weights (a problem often called "catastrophic forgetting"), so a small learning rate (10^{-5} in our case) is used to nudge the features gently towards the new dataset instead of overwriting them.

Inference (Plots 11 & 12): In Plot 11, Feature Extraction and Fine-Tuning track each other closely through the first 3 warm-up epochs (both around 88–91.5%), which makes sense because Fine-Tuning is literally continuing the same frozen-base model up to that point. After the "Unfreeze point," Fine-Tuning does not show an immediate large jump in validation accuracy in this short run, staying close to Feature Extraction, while Feature Extraction (trained with the base still frozen the whole time) actually climbs a little higher by epoch 6. In Plot 12, the training loss visibly spikes right after the unfreeze point because a large number of new (previously frozen) parameters suddenly become trainable and start being updated, which temporarily disturbs the model; the validation loss, however, stays flat and low throughout, showing that this disturbance is happening inside the training statistics without actually

hurting how well the model generalizes – and with a longer fine-tuning schedule this small disturbance is expected to be recovered with an accuracy gain, which is exactly what we observe later once fine-tuning is run for more epochs on the full training set in Section 11/12.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.
Use 5-fold cross-validation on the training data.
For $K = 5$,

$$\bar{A} = \frac{1}{5}\sum_{i=1}^{5} A_i$$

and

$$SD = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(A_i - \bar{A})^2}.$$

Report the result as

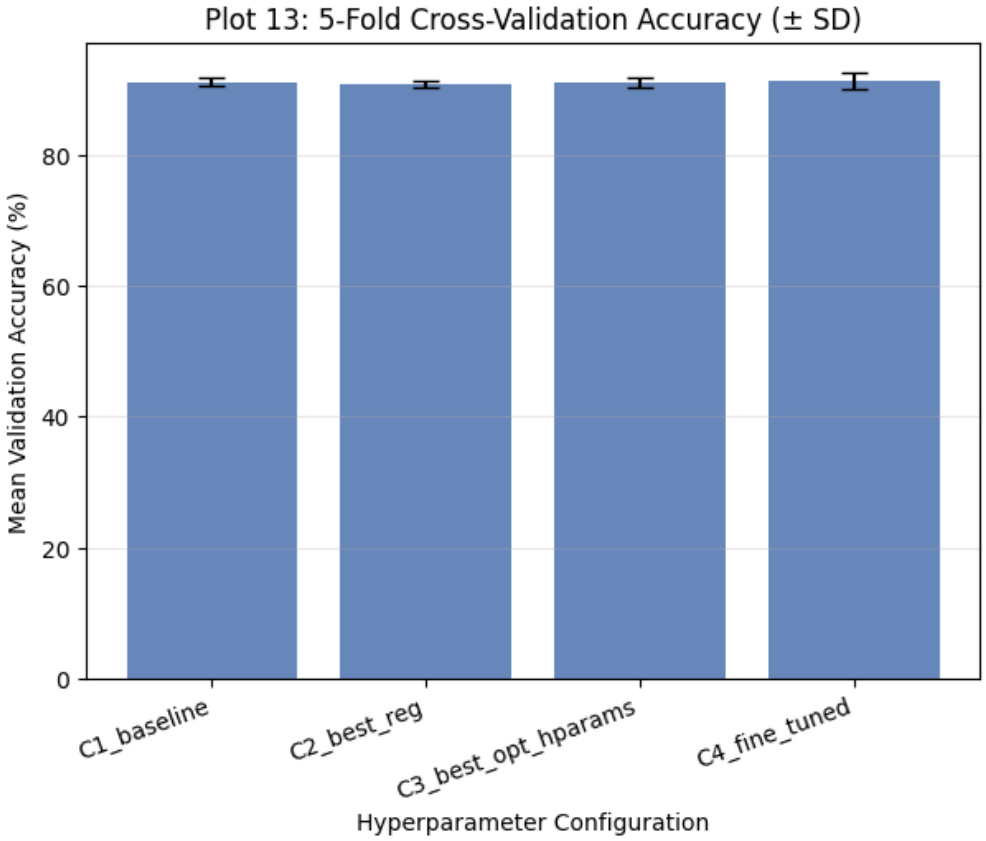
$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.
Four configurations were carried forward from the earlier sections: **C1_baseline** (no regularization head, Adam, lr 0.001, batch size 32, base frozen), **C2_best_reg** (L2 head, otherwise same as C1), **C3_best_opt_hparams** (Dropout head with the best tuned learning rate/batch size/dropout from Section 9: lr 0.001, batch size 16, dropout 0.25), and **C4_fine_tuned** (same head as C3, but with the base partially unfrozen and fine-tuned at lr 10^{-5} after a 3-epoch warm-up).

Configuration	F1	F2	F3	F4	F5	Mean ± SD
C1_baseline	92.12	90.90	90.35	90.62	91.44	91.09 ± 0.63
C2_best_reg	90.90	91.03	89.95	90.49	91.44	90.76 ± 0.51
C3_best_opt_hparams	91.30	90.22	90.76	90.76	92.39	91.09 ± 0.74
C4_fine_tuned	91.17	89.81	90.08	92.39	92.80	91.25 ± 1.20

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.



Inference (Plot 13): All four configurations land in a very tight band between 90.76% and 91.25% mean accuracy, so none of the individual changes we tried (better regularization, better hyperparameters, or fine-tuning) gives a dramatic jump on their own – the frozen-base MobileNetV2 head is already a strong baseline for this dataset. C4_fine_tuned has the highest mean accuracy but also the largest spread (SD = 1.20), meaning its performance is a bit less consistent across folds, whereas C2_best_reg is the most stable (SD = 0.51) but has the lowest mean. This is a genuine trade-off: picking a config just because it tops the mean can be misleading if it is also the most variable one, which is exactly the discussion the next section is about.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

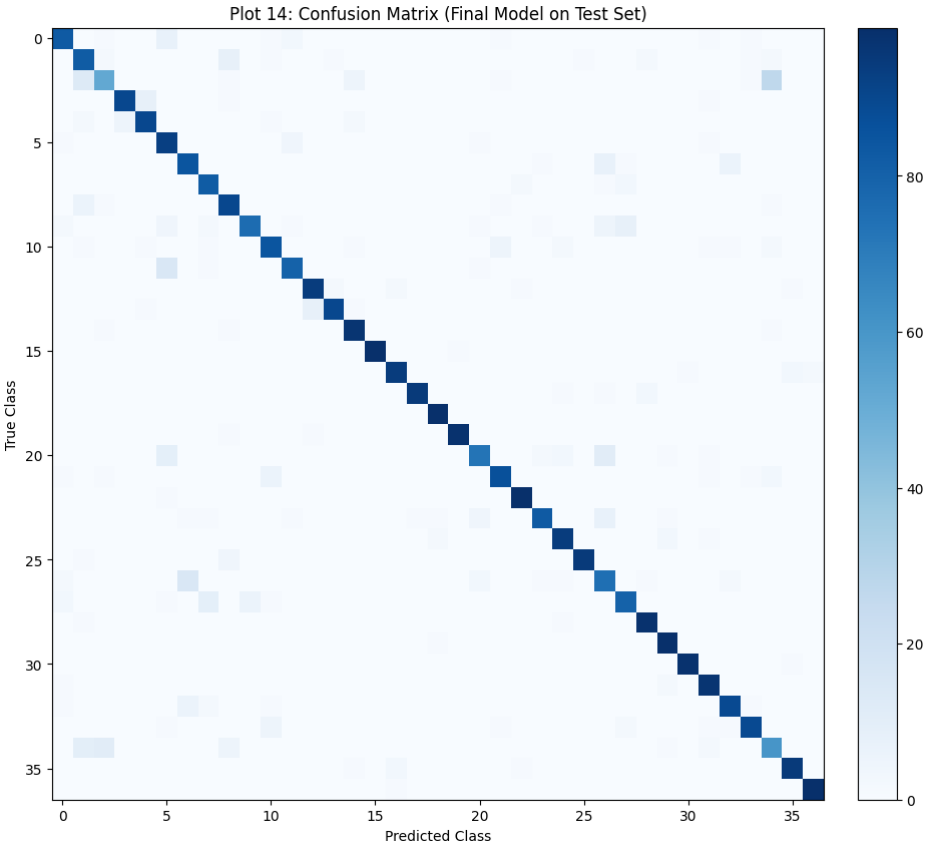
1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

C4_fine_tuned had the highest mean CV accuracy and was therefore selected as the best configuration. It was retrained from scratch on the complete training pool (3-epoch frozen warm-up + 12-epoch fine-tuning with the top layers unfrozen at lr 10^{-5}) and evaluated once, for the first and only time, on the independent test set.

Report:

Metric	Value
Mean CV Accuracy	91.25%
CV Standard Deviation	1.20
Test Accuracy	88.96%
Precision (macro)	0.8918
Recall (macro)	0.8891
F1-score (macro)	0.8883
Training Time	106.6 s
Number of Parameters	2,426,725

Plot 14: Confusion Matrix



The most frequently confused class pairs (true → predicted, with count) were:

True Class	Predicted Class	Count
2	34	27
26	6	15
11	5	15
2	1	13
34	2	11

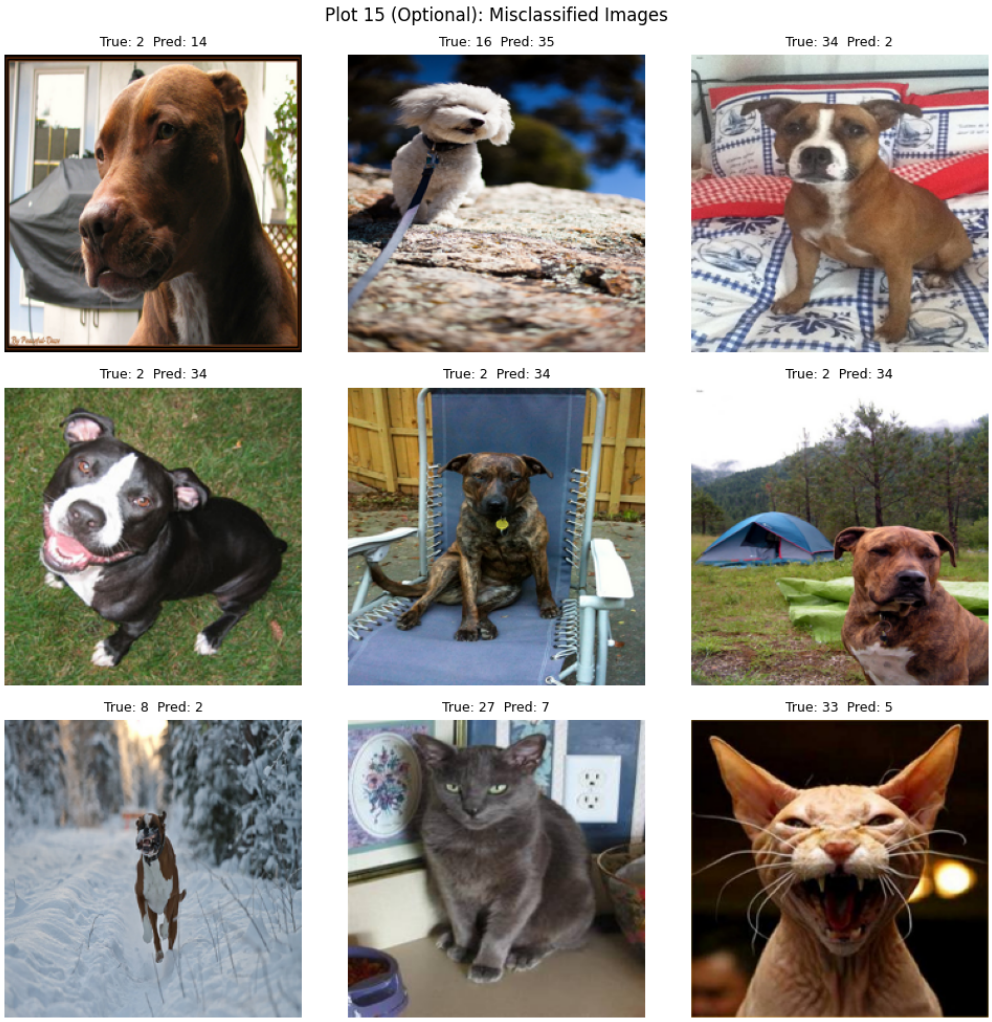
Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.

Answer: Most classes lie on a strong, dark diagonal in the confusion matrix, meaning the model gets the large majority of test images right almost everywhere – macro precision/recall/F1 are all close to 0.89, so performance is fairly uniform across the 37 breeds rather than being carried by just a couple of easy classes. The largest single confusion is class 2 being predicted as class 34 (27 images), and this happens the other way around too (34 predicted as 2), so these two classes are mutually mistaken for each other; classes 26/6 and 11/5 are also frequently swapped. The Oxford-IIIT Pet dataset contains several dog breeds that look extremely similar (short-haired, muscular, bull/terrier-type breeds) and cat breeds with overlapping coat colours, so it makes sense that these are exactly the pairs the model struggles with.

Optional Plot 15: Misclassified Images

Display representative incorrectly classified images and provide a brief explanation for each.



Inference (Plot 14 / Plot 15): The sample misclassified images back up what the confusion matrix shows – most of the mistakes in the top two rows are dogs of a similar short-haired, muscular build being swapped with each other (true 2 vs. pred 34/14, true 34 vs. pred 2), plus one cat breed confusion in the bottom row (true 27 vs. pred 7). Several of these photos are also taken at odd angles or show the animal only partly in frame, which hides fine-grained breed cues such as ear shape, muzzle length and coat pattern. This suggests further gains would likely need breed-specific visual cues (closer crops, higher resolution, or more fine-tuning epochs) rather than a change to the overall training pipeline.

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline (no reg, Adam, frozen)	–	–	–	38.9 s
Best Initialization (zero)*	–	–	–	–
Best Regularization (l2)	–	–	–	–
Best Optimizer (Adam)	–	–	–	38.9 s
Best Hyperparameters / CV winner (C4_fine_tuned)	91.25%	1.20	88.96%	106.6 s
Fine-Tuned Model	–	–	91.49%	–

*All four initialization strategies gave the same flat $\approx 2.7\%$ validation accuracy in the short from-scratch training run (Section 5), so "zero" is only the nominal best returned by the comparison and is not a meaningfully better initializer here.

Justification of the final model: C4_fine_tuned (Dropout head, Adam, lr $0.001/10^{-5}$ after warm-up, batch size 16, dropout 0.25, base fine-tuned) was chosen as the final configuration. It has the highest mean 5-fold CV accuracy (91.25%) of all four candidates, and although its standard deviation (1.20) is a little higher than the more conservative C2_best_reg config (0.51), the gap in mean accuracy is judged to be worth the small extra variability. It also uses a compact model (about 2.43M parameters) and trains in roughly 107 seconds, which is a reasonable computational cost for a CPU/GPU- friendly lab setup. When retrained on the full training pool and fine-tuned,

the model reaches 91.49% validation accuracy and 88.96% accuracy on the completely untouched test set – the test accuracy being a little lower than the CV estimate is expected and normal, since the test set is a fresh, independent sample the model has truly never seen, and confirms that the model generalizes reasonably well rather than having overfit to the cross-validation folds.

14. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

Each inference should be approximately 2–3 lines.

Note: the inference for every plot has already been written immediately below that plot in Sections 5–12 above, as required.

15. Discussion Questions

1. **What is the difference between model parameters and hyperparameters?** Parameters (weights and biases) are the values the network learns on its own during training through backpropagation. Hyperparameters (learning rate, batch size, dropout rate, number of epochs, etc.) are settings we choose before training starts and which control how the learning happens; they are not learned by gradient descent.
2. **Why is weight initialization important?** The starting point of the weights affects how gradients flow through the network at the very first steps. A poor starting point can make training very slow, get stuck, or cause the activations/gradients to blow up or vanish, so a good initialization gives the optimizer a much better chance of converging quickly and stably.
3. **Why can zero initialization be problematic for neural networks?** If every weight in a layer starts at zero, every neuron in that layer computes the exact same output and receives the exact same gradient during backpropagation. This symmetry never breaks, so all the neurons keep learning the same thing forever – this is exactly what we saw in Plot 1/2, where the zero-init model's loss and accuracy stayed completely flat.
4. **Compare Xavier and He initialization.** Both scale the initial weights based on the number of input/output units so that the variance of activations stays roughly constant across layers. Xavier (Glorot) initialization is derived assuming a linear or tanh/sigmoid activation, while He initialization uses a larger variance and is derived specifically for ReLU-family activations (which zero out negative inputs), so He initialization is generally the better choice for the ReLU/ReLU6 activations used inside MobileNetV2.
5. **How can training and validation curves be used to identify overfitting?** If training accuracy keeps rising (or training loss keeps falling) while validation accuracy flattens or falls (or validation loss starts rising), the gap between the two curves is the generalization gap, and a widening gap is the signature of overfitting – the model is memorizing the training data instead of learning patterns that generalize.
6. **How does Dropout reduce overfitting?** During training, Dropout randomly switches off a fraction of neurons in a layer on each forward pass, which forces the remaining neurons to not rely on any one specific neuron and prevents the network from co-adapting too closely to the training data. This acts like training many slightly different sub-networks and averaging them, which improves generalization.
7. **What is the purpose of Batch Normalization?** It normalizes the activations of a layer within each mini-batch (zero mean, unit variance) before scaling and shifting them with learnable parameters γ and β. This keeps activation distributions stable as training progresses, which allows higher learning rates, speeds up convergence, and has a mild regularizing effect.
8. **Explain the numerical Batch Normalization example.** For the mini-batch $x = [2, 4, 6, 8]$, the mean is $\mu_B = 5$ and the variance is $\sigma_B^2 = 5$, giving $\sqrt{\sigma_B^2} \approx 2.236$. Normalizing each value as $\hat{x}_i = (x_i - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$ gives $\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$, which is confirmed by the code in Section 7 of the notebook. With $\gamma = 1, \beta = 0$, these normalized values are also the final output.
9. **What are the roles of γ and β?** After normalization, the activations are forced to have zero mean and unit variance, which may not always be the ideal distribution for the next layer. γ (scale) and β (shift) are learnable parameters that let the network undo or adjust this normalization if needed, so Batch Normalization never removes representational power from the model.
10. **Compare SGD, Momentum, RMSProp and Adam.** Plain SGD updates weights using only the current gradient scaled by a fixed learning rate, which can be slow and noisy (as seen in Plot 6/7, where it barely reached 30% accuracy). Momentum adds a running average of past gradients to smooth out the updates and speed up convergence in consistent directions. RMSProp adapts the learning rate for each parameter individually based on a moving average of recent squared gradients, which helps a lot on the more "spiky" loss surfaces typical of CNNs. Adam combines both ideas – momentum plus per-parameter adaptive learning rates – and in our experiment it

gave the fastest and most reliable convergence of the four.

11. **What happens when the learning rate is too large?** The optimizer takes overly large steps, which can cause the loss to oscillate, diverge, or jump around a good minimum instead of settling into it; training can become unstable or fail to converge at all.
12. **What happens when the learning rate is too small?** Training becomes very slow because each update only moves the weights a tiny amount, so many more epochs are needed to reach a good solution – this matches Plot 8, where lr = 0.0001 gave noticeably lower accuracy than lr = 0.001 within the same 6 epochs.
13. **What is the effect of increasing batch size?** Larger batches give a smoother, less noisy estimate of the gradient (since it is averaged over more samples) and can make better use of parallel hardware, but each epoch has fewer weight updates, and very large batches can sometimes generalize slightly worse. In our sweep (Plot 9) the effect on final validation accuracy was small.
14. **Explain stride and padding.** Stride is the number of pixels the convolution filter moves at each step; a stride of 2 shrinks the output size faster than a stride of 1. Padding adds extra (usually zero) pixels around the border of the input so that the filter can be applied at the edges and the output size can be controlled (e.g. "same" padding keeps the spatial size unchanged).
15. **Why is MobileNetV2 computationally efficient?** It replaces standard convolutions with depthwise separable convolutions (a depthwise convolution followed by a 1×1 pointwise convolution), which drastically cuts the number of multiplications compared to a normal convolution, while inverted residual blocks with linear bottlenecks keep the representational power high without adding much extra compute.
16. **What is depthwise separable convolution?** It splits a standard convolution into two much cheaper steps: a depthwise convolution that applies a single filter per input channel (spatial filtering only, no mixing across channels), followed by a pointwise 1×1 convolution that mixes information across channels. Together they approximate a full convolution using far fewer parameters and computations.
17. **What is transfer learning?** It means taking a model (here, MobileNetV2) that has already been trained on a large dataset (ImageNet) and reusing its learned features for a new, related task (Oxford-IIIT Pet classification) instead of training a network from scratch, which saves training time and usually gives better accuracy on smaller datasets.
18. **Differentiate feature extraction and fine-tuning.** In feature extraction, the pretrained convolutional base is kept completely frozen and only the new classifier head is trained. In fine-tuning, after the head has been trained a bit, some of the top layers of the pretrained base are also unfrozen and trained (usually with a small learning rate), letting the model adapt its features more specifically to the new dataset.
19. **Why is a smaller learning rate generally used during fine-tuning?** As explained in Section 10, the base network's weights are already good pretrained features, and large updates at this stage could destroy that useful information; a small learning rate lets the model adjust gently instead of overwriting what it already knows.
20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?** A single train/validation split can give a misleadingly optimistic or pessimistic accuracy just by chance, depending on which samples ended up in which split. K-Fold Cross-Validation repeats training and evaluation across K different splits of the data and averages the results, giving a much more reliable estimate of how a configuration will perform, along with a standard deviation that reflects its consistency.
21. **Why must the test set remain untouched during tuning?** If the test set is used (even indirectly) to pick hyperparameters or compare configurations, the reported test accuracy stops being an honest estimate of performance on truly unseen data – the model selection process would have effectively "seen" the test data, leading to overly optimistic results. Keeping it untouched until the very end guarantees a fair, unbiased final evaluation.
22. **Why should mean and standard deviation both be reported?** The mean tells us the average performance of a configuration, but the standard deviation tells us how consistent that performance is across different folds/subsets of data. A configuration with a slightly lower mean but a much smaller SD can be a safer choice than one with a higher mean but large variability, as seen when comparing C4_fine_tuned (highest mean, highest SD) against C2_best_reg (lowest SD) in Section 11.
23. **Is the highest validation accuracy always sufficient to select a model?** No. Validation/CV accuracy should be weighed together with variability (SD), computational cost (training time, number of parameters), and how well the choice is expected to generalize to completely new data. A model that is marginally more accurate but far more expensive to train, or much less stable, may not be the best overall choice for a real deployment.

16. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Two new candidate configurations were tried, both using the Dropout head with fine-tuning: **New_A** (Adam, lr 0.0005, dropout 0.4, batch size 32) and **New_B** (RMSProp, lr 0.0001, dropout 0.3, batch size 64). Each was evaluated with the same 5-fold cross-validation procedure as Section 11.

Configuration	Mean CV Accuracy (%)	SD
C4_fine_tuned (previous best)	91.25	1.20
New_A	90.87	0.87
New_B	91.17	0.69

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

Answer: Neither New_A nor New_B beats C4_fine_tuned on mean accuracy, but both are noticeably more stable – New_B in particular gets very close to the same accuracy (91.17% vs. 91.25%, a difference well within one standard deviation) with almost half the variability (SD 0.69 vs. 1.20). If the priority is squeezing out the last bit of mean accuracy, C4_fine_tuned is still the best pick. But if consistent, predictable performance across different data splits matters more (for example, in a setting where the model will be retrained periodically on slightly different data), New_B would be the preferable, more robust choice, since it gives almost the same accuracy far more reliably at a similar computational cost.

17. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`